

FPGA-Accelerated Custom Log2 and Square Root Instructions for a Pipelined RV32IM RISC-V Processor

Zhou, Yuchen

yzhou276@jh.edu

Introduction

This project is motivated by the need to improve the execution speed of arithmetic operations that are commonly expensive when implemented purely in software. Operations such as integer square root and logarithm base two can require many instructions when executed using standard RV32IM arithmetic instructions, especially when they are implemented as iterative software routines. By adding custom hardware-supported instructions for `sqr` and `log2`, these computations can be accelerated directly in the processor datapath, reducing instruction count and improving overall execution latency. Another important motivation of this project is to connect algorithm development with real FPGA implementation. The square root and `log2` algorithms are first modeled in MATLAB to verify their numerical behavior, expected latency, and functional correctness. After that, the validated models are translated into FPGA RTL, integrated into the pipelined RISC-V processor, and verified through simulation by comparing RTL results against the MATLAB reference model. This workflow provides a complete path from high-level algorithm exploration to hardware implementation and verification, demonstrating how custom ISA extensions can improve computational performance in an FPGA-based RISC-V system.

SQRT and LOG2 software analysis

Pseudo code for $\text{ceil}(\log_2(n))$

Function CEIL_LOG2(n):

```
if n <= 1:
    return 0

value = n - 1
result = 0

while value > 0:
    value = value >> 1
    result = result + 1

return result
```

In Software, the $\log_2(n)$ operation is typically implemented using an iterative bit-shifting approach like the pseudo code. The main idea is to determine how many bits are required to represent the value $n-1$. The subtraction by one is important because exact powers of two should not be rounded up. For example, $\text{ceil}(\log_2(8))$ should return 3, not 4. After computing $n - 1$, the algorithm repeatedly shifts the value to the right until the value becomes zero. Each right shift removes one bit from the number, and the loop counter records how many shifts were required. This final count represents the ceiling of the base-2 logarithm.

Although this algorithm is simple, it introduces variable execution latency when implemented in assembly. For a 32-bit input, the loop may execute up to 32 times. Each loop iteration requires multiple assembly

instructions, including a comparison, conditional branch, shift operation, increment operation, and jump back to the loop. As a result, the total latency is not just the number of loop iterations, but the number of iterations multiplied by the instruction cost of each loop body. Branch instructions may also disturb the processor pipeline, causing additional delay. Therefore, even a simple mathematical operation like $\log_2$ can become relatively expensive when implemented only with standard RV32IM instructions.

Pseudo code for *floor(sqrt(n))*

Function FLOOR_SQRT(n):

```

root      = 0
remainder = 0

for i from 15 down to 0:

    root = root << 1

    remainder = (remainder << 2) | ((n >> (2*i)) & 0b11)

    trial = (root << 1) | 1

    if remainder >= trial:
        remainder = remainder - trial
        root = root | 1

return root

```

The *floor(sqrt(n))* operation is more computationally expensive because it uses an iterative digit-by-digit square root algorithm. For a 32-bit unsigned input, the square root result is at most 16 bits wide, so the algorithm performs 16 iterations. Each iteration processes two bits of the input number and determines one bit of the square root result. The algorithm maintains a partial root and a remainder. During each iteration, the next two input bits are shifted into the remainder, a trial subtraction value is generated from the current root, and the algorithm decides whether the next root bit should be 1 or 0.

When this square root algorithm is converted into assembly, each iteration requires several low-level instructions. These include shifts, masks, logical OR operations, comparisons, conditional branches, subtraction, loop-index updates, and branch-back instructions. Since the loop runs 16 times for a 32-bit input, the total instruction count becomes high. In a pipelined processor, conditional branches inside the loop can also create control hazards, and data dependencies between *root*, *remainder*, and *trial* limit how much the execution can be overlapped. Therefore, even though the algorithm is efficient from a mathematical perspective, its software implementation still has high latency on a basic RV32IM processor.

Approach

Hardware-Accelerated Algorithm

Log2

The $\log_2$ calculation can be mapped to priority detection logic that directly finds the highest set bit of $N-1$. This allow the result to be generated into a single combinational pass, or in one clock cycle if registered as an ALU-

S26 Computer Architecture – Log2 and Square Root ISA Extension on RISC-V CPU style custom instruction. As a result, the operation is reduced from tens or hundreds of clock cycles using the software method to approximately one hardware instruction cycle.

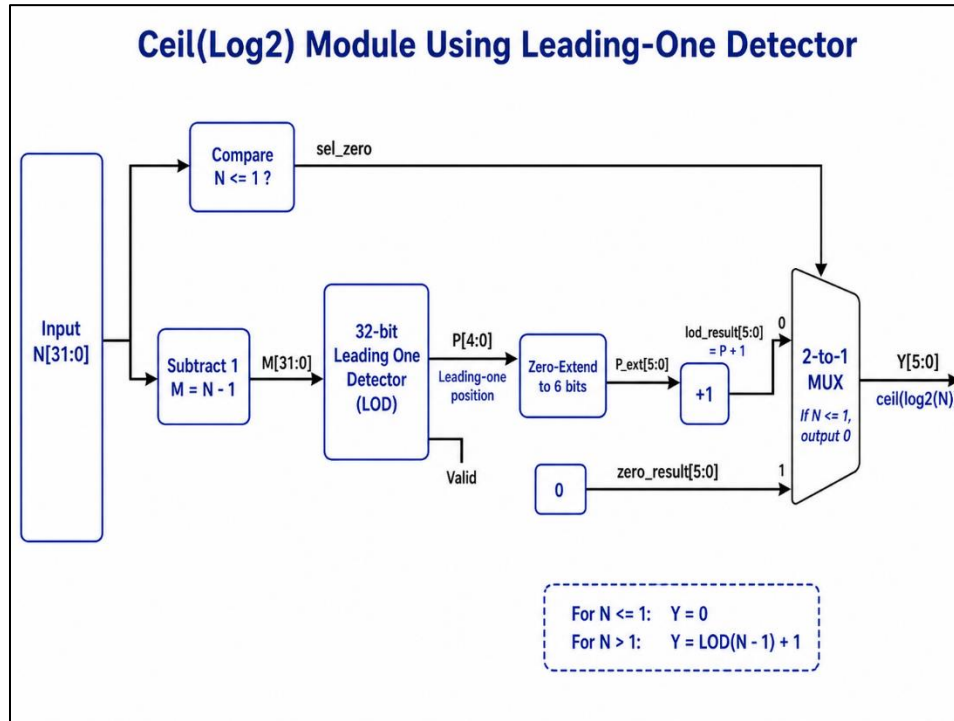


Figure 1. Log2 Leading One Detection Algorithm

Square Root

For the custom `floor(sqrt(N))` instruction, several hardware algorithms were considered, including restoring, non-restoring, Newton-Raphson, and CORDIC. Newton-Raphson can converge in fewer iterations, but each iteration is more complex because it depends on division or reciprocal operations and also requires a good initial estimate. This makes the control and datapath harder to implement and verify in a small FPGA-based processor extension. In comparison, the non-restoring square root algorithm is more hardware-friendly because it avoids division and computes the result one bit at a time using mainly shift, add/subtract, and comparison operations. Although non-restoring square root has higher iteration latency than Newton-Raphson, its simple and predictable datapath makes it easier to integrate as a custom multi-cycle instruction. For a 32-bit input, the algorithm typically requires 16 iterations because each step resolves one bit of the square root result. This provides a good balance between hardware complexity, verification effort, and acceleration compared with a pure RV32I software implementation.

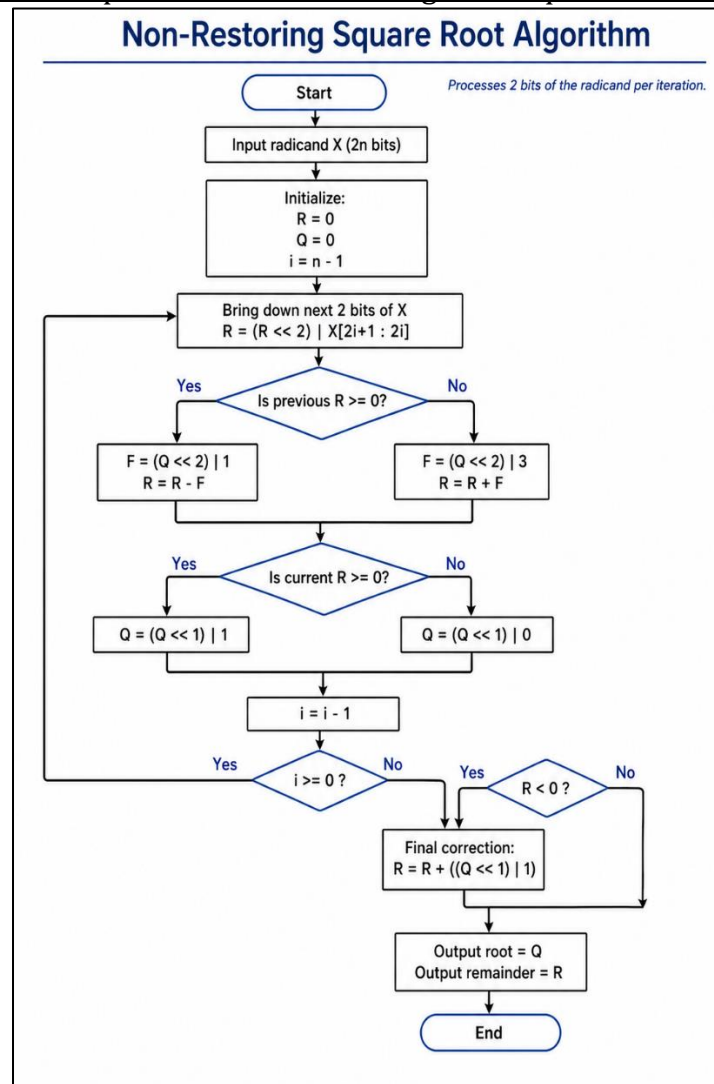


Figure 2. Square Root Non-Restoring Algorithm

RISC-V Pipelined CPU Modification

To support `log2` and `sqr` instruction on the RISC-V CPU, we need to modify the existing architecture by adding control signals at different stages and pipelining the appropriate signals.

Log2

`log2` is a single-cycle, ALU-resident operation. Conceptually it acts like any other R-type ALU op (add, xor, etc.), so the existing pipeline absorbs it almost for free.

- Decode (ID) generates a new `i_log2` flag and selects a previously unused ALU control code for it.
- Operand path is the standard one: `rs1` is read from the `regfile` and forwarded by the existing forwarding muxes — no new bypass.
- Execute (EXE) sees the special ALU code and the ALU's combinational `ceil_log2` block produces the result the same cycle. No stall, no new mux beyond the ALU's internal case.
- MEM and WB are completely unaware; the result rides the existing `ea1` → `ma1` → `wres` writeback path back to the `regfile`.

Square Root

The custom `sqr` instruction is implemented as a 16-cycle iterative hardware accelerator. Since the operation cannot complete in a single execute stage cycle, it must temporarily stall the pipeline while the square root unit is computing the result. Instead of adding a completely separate stall mechanism, the design reuses the existing multi-cycle control infrastructure already used by the RV32M multiply/divide unit.

In the **instruction decode** stage, a new `i_sqr` decode signal is added to identify the custom square root instruction. From this signal, the control unit generates two related outputs: `sqr` and `start_sqr`. The `sqr` signal remains active while a square root instruction is present in the pipeline, while `start_sqr` is a one-cycle pulse used to launch the iterative square root accelerator. The instruction is also included in the register writeback control logic so that the final result is written to `rd`. Since the square root operation only uses `rs1` as its input, `rs1` is also included in the forwarding logic to ensure the accelerator receives the most recent operand value.

Across the **ID/EX** pipeline boundary, the `sqr` control signal is registered into `esqr` using the same pipeline register enable signal used by the other execute-stage control bits. When the pipeline is stalled during the square root operation, this enable is held low, allowing `esqr` to remain asserted for the entire 16-cycle execution window. The `start_sqr` pulse, however, is passed directly into the execute stage as `estart_sqr` rather than being registered. This allows the accelerator to receive only a single start pulse, which is sufficient to begin the iterative computation.

In the **execute** stage, the square root accelerator is connected in parallel with the existing ALU and RV32M multiply/divide unit. The radicand input to the accelerator comes from the forwarded `rs1` operand, using the same `ea` datapath already available to the ALU. The execute-stage result mux is extended so that the square root result has priority over the standard ALU and RV32M results. When `esqr` is asserted, the lower 16-bit square root output is zero-extended to 32 bits and selected onto `ea1`.

The pipeline stall behavior is also integrated with the existing multi-cycle wait signal. The design allows the pipeline to advance only when the multiply/divide unit is ready and either no square root instruction is active in the execute stage or the square root unit has completed. While the square root accelerator is busy, the existing wait logic freezes the pipeline by holding the normal pipeline write-enable signals inactive. When the square root unit asserts its done signal, the wait condition is released, and the computed result is allowed to move through the normal **EX/MEM** and **MEM/WB** pipeline registers.

No additional logic is required in the memory or writeback stages. After the square root result is selected onto the execute-stage result bus, it follows the same path as a normal ALU result. The **EX/MEM** register captures the result, and the writeback stage writes it into the destination register `rd`. This keeps the multi-cycle square root instruction compatible with the existing pipelined processor structure while minimizing the amount of new control logic required.

Custom Instruction Opcode

Instruction	Name	Format	Opcode	funct3	funct7	Description
<code>log2</code>	Logarithm Base 2	R-type	0b0110011	0x0	0x08	Computes the base-2 logarithm of an integer
<code>sqr</code>	Square Root	R-type	0b0110011	0x1	0x08	Computes the square root of an integer

Figure 3. Custom R-type instruction encoding for `log2` and `sqr`

This table defines the custom instruction encodings used for the proposed RV32IM extension. Both `log2` and `sqr` are assigned as R-type instructions because they operate on register operands and write the result back

S26 Computer Architecture – Log2 and Square Root ISA Extension on RISC-V CPU to a destination register. The same opcode `0b0110011` is used to keep the instructions in the standard R-type ALU instruction format, while different funct3 values are assigned to distinguish between the two operations. The shared funct7 = `0x08` marks these operations as custom arithmetic functions and separates them from existing RV32IM instructions.

```

sqrt rd, rs1, x0    -> rd = isqrt(rs1)
log2 rd, rs1, x0    -> rd = ceil(log2(rs1))

```

Figure 4. Assembly syntax and functional behavior of custom sqrt and log2 instructions

This figure shows the assembly-level format of the proposed custom instructions. The sqrt instruction computes the integer square root of rs1, while the log2 instruction computes `ceil(log2(rs1))`. Since both operations only require one input operand, rs2 is set to x0 as a placeholder to preserve the R-type instruction format. This allows the custom instructions to be decoded consistently with the existing ALU datapath while adding dedicated hardware acceleration for expensive mathematical operations.

Implementation

Please visit the project repository at <https://github.com/yzhou276/Computer-Architecture-Project>

Hardware Accelerated Module

I followed the MATLAB model to FPGA RTL design flow to develop the log2 and sqrt modules. In the report, I am showing the MATLAB and RTL simulation of the non-restoring square root module. Please visit the project repository to view the MATLAB model and RTL code of the log2 module.

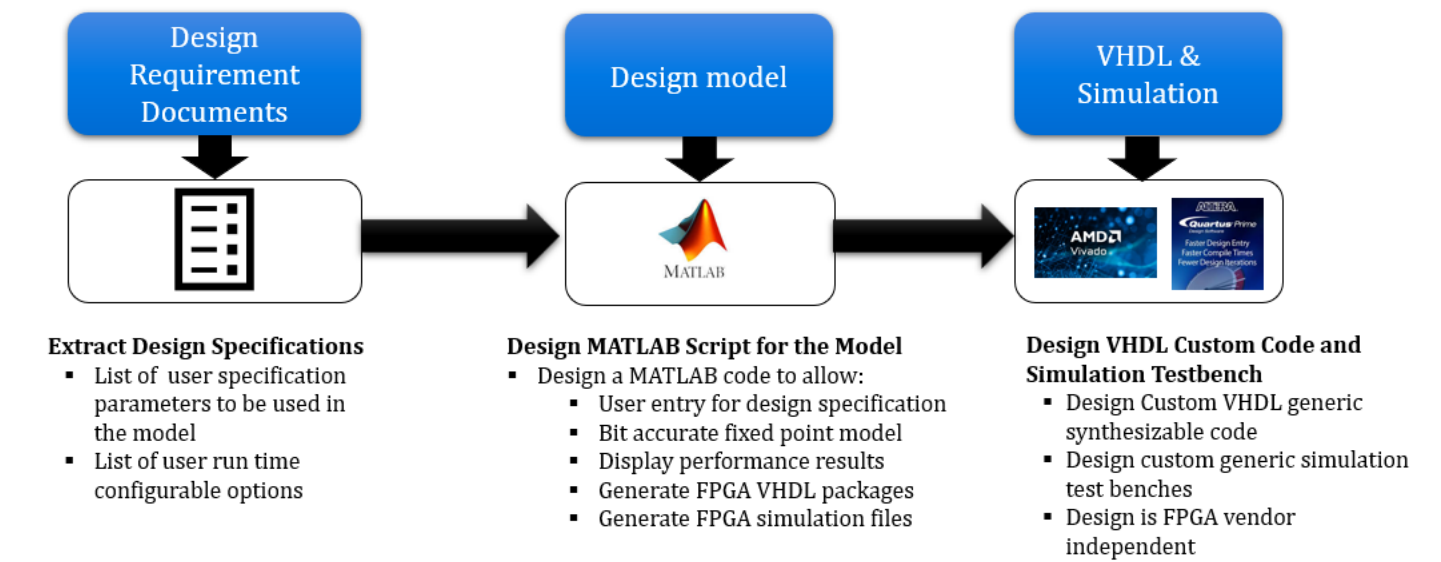


Figure 5. MATLAB model to FPGA RTL Design Flow

Non-restoring sqrt32 RTL

```

module isqrt32_nonrestoring (
    input logic      clk,
    input logic      rst_n,
    input logic      start,    // Pulse high for one cycle to start a new sqrt operation
    input logic [31:0] radicand, // 32-bit unsigned input operand
    output logic      busy,    // High while the iterative algorithm is running
    output logic      done,    // Pulses high for one cycle when result is ready
    output logic [15:0] root,  // floor(sqrt(radicand))

    // radicand - root^2
    output logic [16:0] remainder
);

// 32-bit radicand -> 16 groups of 2 bits -> 16 iterations
localparam int NUM_ITER = 16;
// Shift register holding the remaining input bit-pairs to process
logic [31:0] x_reg;
// Partial / final square-root result
logic [15:0] q_reg;
// Signed partial remainder used by the non-restoring algorithm
// Width is chosen large enough for the intermediate add/sub values
logic signed [18:0] p_reg;
// Counts remaining iterations
logic [4:0] iter_reg;
// Next 2-bit group pulled from the MSBs of x_reg each cycle
logic [1:0] next2;
// Partial remainder after shifting left by 2 and appending next2
logic signed [18:0] p_shifted;

// Trial term:
//   if P >= 0, use (Q<<2)+1
//   if P < 0, use (Q<<2)+3
logic signed [18:0] f_val;
// Next partial remainder after add/subtract
logic signed [18:0] p_next;
// Next partial root after appending one new root bit
logic [15:0] q_next;

// Grab the current top 2 bits of the radicand shift register
assign next2 = x_reg[31:30];
// Shift the current partial remainder left by 2
// and append the next 2-bit group from the radicand
assign p_shifted = (p_reg <<< 2) + $signed({17'd0, next2});

always_comb begin
    // Non-restoring rule:

```



```

// If previous partial remainder is nonnegative, subtract.
// If previous partial remainder is negative, add.
if (p_reg >= 0)
    f_val = $signed({1'b0, q_reg, 2'b01}); // (Q << 2) + 1
else
    f_val = $signed({1'b0, q_reg, 2'b11}); // (Q << 2) + 3

// Perform the non-restoring add/sub step
if (p_reg >= 0)
    p_next = p_shifted - f_val;
else
    p_next = p_shifted + f_val;

// Append the next root bit based on the sign of the new remainder
// Nonnegative -> next root bit = 1
// Negative -> next root bit = 0
if (p_next >= 0)
    q_next = {q_reg[14:0], 1'b1};
else
    q_next = {q_reg[14:0], 1'b0};
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Asynchronous reset
        x_reg    <= '0;
        q_reg    <= '0;
        p_reg    <= '0;
        iter_reg <= '0;
        busy     <= 1'b0;
        done     <= 1'b0;
        root     <= '0;
        remainder <= '0;
    end
    else begin
        // done is a one-cycle pulse
        done <= 1'b0;

        // Start a new operation only when the unit is idle
        if (start && !busy) begin
            x_reg    <= radicand;           // load full 32-bit input
            q_reg    <= 16'd0;              // partial root starts at 0
            p_reg    <= 19'sd0;             // partial remainder starts at 0
            iter_reg <= NUM_ITER[4:0];      // 16 iterations total
            busy     <= 1'b1;
        end
        else if (busy) begin

```



```

// Consume the current top 2 bits and shift next pair into place
x_reg <= {x_reg[29:0], 2'b00};

// Update partial root and partial remainder
q_reg <= q_next;
p_reg <= p_next;

// iterate_reg counts down from 16 to 0, and when it reaches 1 we are on the
last iteration
iter_reg <= iter_reg - 1'b1;

// Last iteration: capture outputs and finish
if (iter_reg == 5'd1) begin
    busy <= 1'b0;
    done <= 1'b1;
    root <= q_next;

    // Final correction for non-restoring algorithm:
    // if the final partial remainder is negative,
    // restore it once to get the true nonnegative remainder
    if (p_next < 0)
        remainder <= p_next + $signed({1'b0, q_next, 1'b1});
    else
        remainder <= p_next[16:0];
end
end
end
end
endmodule

```

RTL Simulation Result and MATLAB Verification

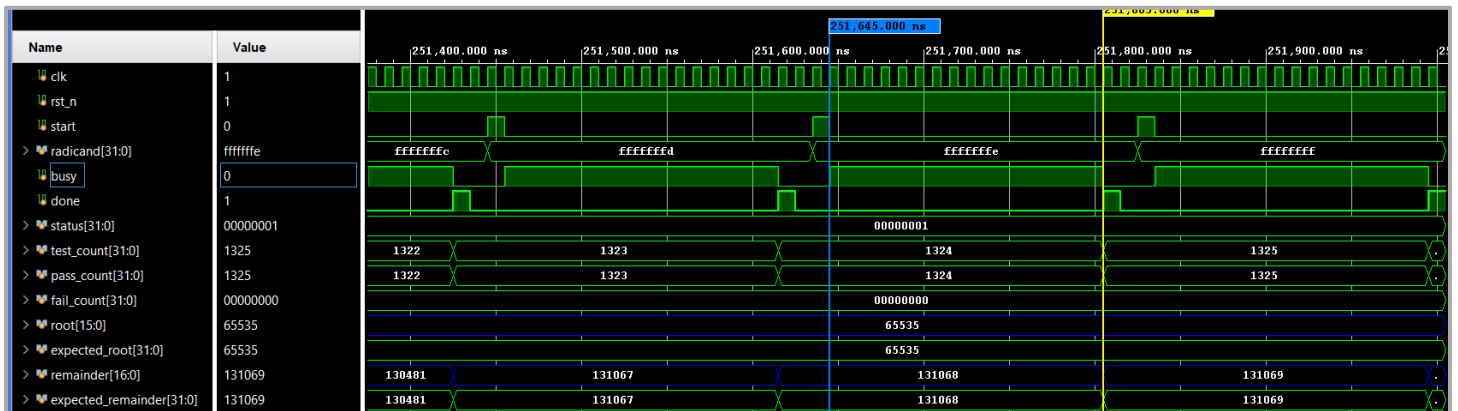


Figure 6. SQRD – RTL Simulation

S26 Computer Architecture – Log2 and Square Root ISA Extension on RISC-V CPU

The image shows a MATLAB Editor window with the file 'matlab2rtl_cmp.m' open. The script compares SV non-restoring integer sqrt results against MATLAB reference. The Command Window displays the following output:

```

Command Window
Total tests      : 1326
Root mismatches  : 0
Remainder mismatches : 0
Total vector mismatches: 0
All RTL non-restoring sqrt results match MATLAB reference.
fx >>
  
```

Figure 7. SQRT – Compare MATLAB output to RTL output

The image shows the MATLAB Command Window with the following output:

```

Command Window
Total tests      : 1326
Root mismatches  : 0
Remainder mismatches : 0
Total vector mismatches: 0
All RTL non-restoring sqrt results match MATLAB reference.
fx >>
  
```

Figure 8. SQRT – Comparison Report

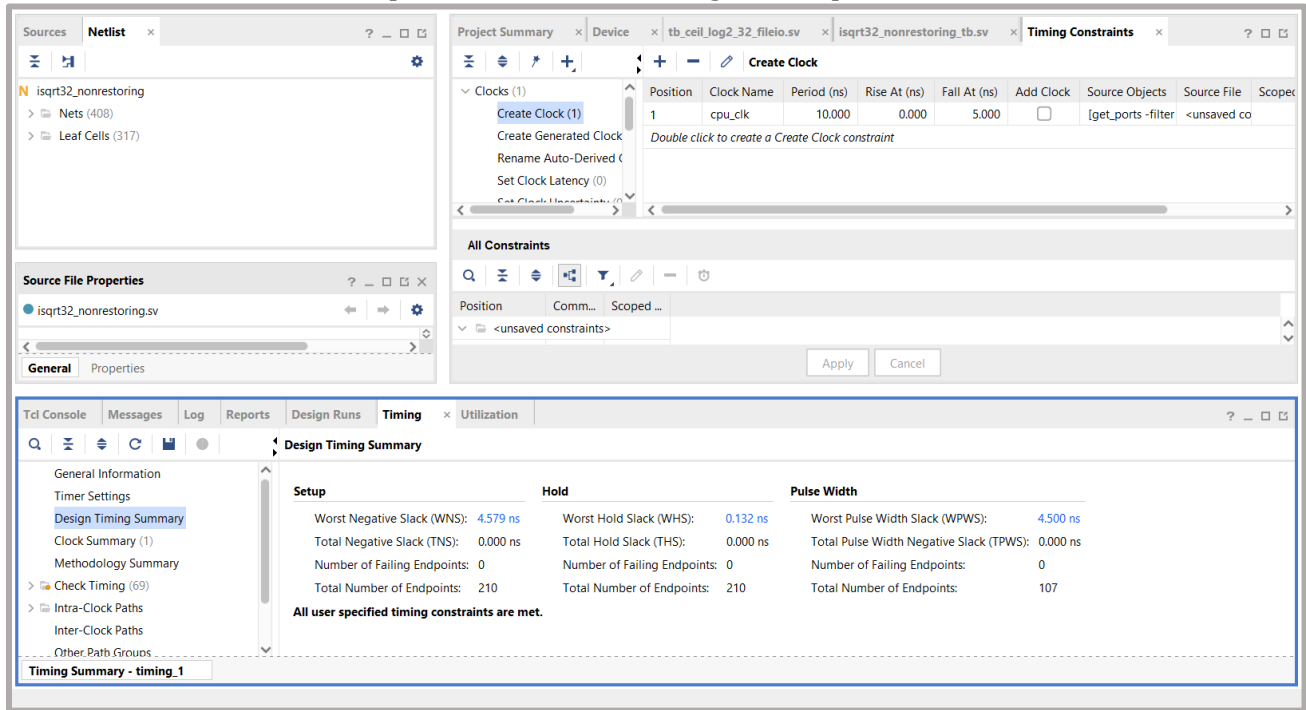


Figure 9. Sqrt – Static Timing Analysis

Name	Slice LUTs (63400)	Slice Registers (126800)
N isqrt32_nonrestoring	85	106

Figure 10. Sqrt – FPGA Source Utilization

RISC-V CPU Integration

After successfully verifying the standalone non-restoring integer square root module and the leading-one detection based log2 module, I integrated both hardware accelerators into the existing pipelined RISC-V CPU. The CPU was updated according to the changes described in the [RISC-V Pipelined CPU Modification](#) section, which included adding the new computational modules, expanding the instruction decode logic, and routing the required control and data signals through the pipeline.

In addition to integrating the custom hardware instructions, I also implemented the conditional branch instruction **BLE** to support the software-based square root implementation used for performance comparison. This branch instruction was needed because the software square root algorithm relies on loop control and comparison operations. Adding BLE allowed the baseline RV32IM software implementation to run correctly on the modified CPU, enabling a fair comparison between the software and hardware-accelerated approaches.

Overall, this integration step connected the verified arithmetic modules to the full CPU pipeline, updated the control logic, and added the necessary instruction support to evaluate both custom hardware execution and equivalent software execution on the same RISC-V processor.

Performance

Simulation testcases and results are documented in the accompanying [presentation](#). These results provide a clearer view of the performance metrics used to evaluate the custom instructions.

For testing, I wrote manual test cases that load register A from a fixed memory location and then execute the custom instruction to compute $\text{regB} = \log_2(\text{regA})/\text{sqrt}(\text{regA})$. The latency was measured as the number of cycles between the update of regA and the corresponding update of regB.

Because the compiler does not recognize the custom `log2` and `sqrt` instructions, both instructions had to be manually encoded into machine code for simulation. This is also why the test program was intentionally kept simple: a straightforward loop makes it easier to verify the instruction encoding, observe register updates, and measure latency accurately.

An example test sequence for the custom `sqrt` instruction is shown below (same process for `log2`):

```
bf8005b7    // | 10111111100000000000 (bf800) 01011 (rd = x11/a1) 0110111
lui  a1, 0xbf800,      a1 = 0xbf800000
00458593    // | 000000000100 (4) 01011 (rs1 = x11/a1) 000 01011 (rd = x11/a1) 0010011
addi a1, a1, 4,        a1 = 0xbf800004
0005a283    // | 000000000000 (0) 01011 (rs1 = x11/a1) 010 00101 (rd = x5/t0) 0000011
lw    t0, 0(a1),       t0 = Mem[a1 + 0]
bf800537    // | 10111111100000000000 (bf800) 01010 (rd = x10/a0) 0110111
lui  a0, 0xbf800,      a0 = 0xbf800000
10029333    // | 0001000 00000 (rs2 = x0)      00101 (rs1 = x5/t0) 001 00110 (rd = x6/t1)
0110011
sqrt  t1, t0, x0
00652023    // | 0000000 00110 (rs2 = x6/t1) 01010 (rs1 = x10/a0) 010 00000 0100011
sw    t1, 0(a0),       Mem[a0 + 0] = t1
fe9ff0ef    // | J-type imm = -24, rd = x1/ra, opcode = 1101111
jal   ra, next
```

1. Build the switch input address 0xbf800004
2. Read the switches into t0
3. Build the LED output address 0xbf800000
4. Compute `sqrt(t0)` into t1
5. Write the `sqrt` result t1 to the LEDs
6. Jump back and repeat forever

LOG2

Implementation Type	Estimated Latency (# clocks)
FPGA Module	1-2
RV32IM Software	$\sim 4 \times N$ (N = bit-width)

$$\text{Speedup} = \frac{(\text{Software latency})}{(\text{Hardware latency})} \approx \frac{4 * N}{1} \approx 4N, N = \text{minimum \# bits to represent the input}$$

The software implementation of the $\log_2$ operation has latency that scales with the magnitude (or bit-width) of the input value. Specifically, the algorithm repeatedly shifts the input right until it becomes zero, requiring up to N iterations for an N -bit operand in the worst case. Therefore, the execution time increases linearly

with input size. In contrast, the custom FPGA implementation computes $\log_2$ using a leading-one detector, resulting in a fixed and constant latency independent of the input value.

SQRT

Implementation Type	Estimated Latency (# clocks)
FPGA Module	19
RV32IM Software	250

$$\text{Speedup} = \frac{(\text{Software latency})}{(\text{Hardware latency})} \approx 13.2$$

The square root latency scales linearly with operand bit-width ($\approx N/2$ iterations for an N-bit input), whereas the FPGA module has fixed latency, leading to increasing speedup for larger inputs.

Learning and Future Effort

Through this project, I gained a better understanding of RISC-V processor architecture, especially how instructions are decoded, executed, and written back in a pipelined CPU. Adding the custom `log2` and `sqr` instructions required changes across multiple parts of the processor, including instruction decoding, control signal generation, operand forwarding, execute-stage result selection, multi-cycle stall control, and writeback handling. This helped me better understand how an instruction moves through the pipeline and how custom hardware can be integrated into an existing datapath.

One important future effort is to expand the project beyond the custom instruction datapath and connect the processor more closely with memory-mapped peripherals. The current test cases are intentionally simple and mainly focus on proving that the custom instructions work correctly in simulation. A more complete system-level design would include additional memory-mapped interfaces so the processor can load test inputs, store computed results, and communicate with external devices. This would make the verification environment closer to a real FPGA-based embedded system.

Another future improvement is to perform more comprehensive verification after full system integration. The current verification approach uses manually written test cases and waveform inspection to measure the latency between input register updates and output register updates. In the future, I would like to build a larger automated verification flow. One possible approach is to use the MATLAB model to generate a representative set of input values, including edge cases, powers of two, non-powers of two, small values, and large 32-bit values. These test values could be preloaded into an offset region of data memory. The RISC-V program would then iterate through the list, compute both `sqr(N)` and `ceil(log2(N))` for each input, concatenate or pack the results into a compact output word, and store the results into another offset region of data memory. Since the square root result requires at most 16 bits and the `log2` result only requires a small number of bits, the combined result can fit within a 32-bit word.

After simulation-level verification, the next major step is to synthesize and implement the fully integrated system on the FPGA. This includes checking resource utilization, reviewing timing reports, and resolving timing closure issues in the current design. The custom `log2` instruction is based on leading-one detection logic, which may create a long combinational path if implemented as a single-cycle instruction. The `sqr` instruction is multi-cycle, but its control and stall logic must still be checked carefully after integration. Timing analysis will help determine whether additional pipelining, register balancing, or logic restructuring is needed.

S26 Computer Architecture – Log2 and Square Root ISA Extension on RISC-V CPU

A more advanced future design would add a FIFO-buffered UART peripheral at a memory-mapped address. The UART transmit and receive signals could be routed to a PMOD connector on the FPGA and connected to a computer using a low-cost TTL-to-USB serial dongle. With this setup, the processor could send test inputs and computed results to a host computer through the serial interface. A simple Python program could then read the serial data, save the received test samples and results, and compare them against the MATLAB golden reference model. This would provide an end-to-end verification flow from MATLAB test-vector generation, to FPGA execution, to host-side result checking. It would also make the project more practical because the custom instructions could be validated not only in simulation, but also on real hardware.

Reference

- An Efficient Implementation of the Non-Restoring Square Root Algorithm in Gate Level
- Parallel-array implementations of a non-restoring square root algorithm
- Non-Restoring Algorithm for Square Root

Project Repository

- <https://github.com/yzhou276/Computer-Architecture-Project>